Learning a Neural Calculator - Project Report

Filippo Calandra, Federico Disi, Mattia Simone Pietro Mazzonna

October 19, 2026

1 Aim of the project and contributions

The following project aims to develop two separate Machine Learning Models which are able to read an arithmetic expression represented as a string and predict its outcome. The expressions are built over the alphabet `0,1,2,3,4,5,6,7,8,9,+,-,(,),` where each number is a single decimal digit. Therefore, the target is an integer output value. The main objective is to train models that are robust enough to deal with expressions outside the training distribution, such as unusual formula shapes, heavier use of parentheses, different subtraction patterns and longer sequences. As for the individual contributions, Mattia Mazzonna focused on the implementation of the Convolutional Neural Network Model, Filippo Calandra developed the Transformer Model, in collaboration with Federico Disi, cured the "informatics part" of the project (download and upload of the parameters, prompt interface) and coordinated the work, and Federico Disi contributed to the implementation of the Transformer Model, worked on merging the two methods into a single final Colab notebook, and performed the analysis and comparison of the results obtained by both models. All three group members took part in writing the final report. Artificial intelligence was used as a supportive tool to generate some code, help us fix programming errors, and make the source code as neat as possible.

2 Preliminary dataset analysis

The dataset is divided into five predefined splits as follows:

Split	**Train**	**Validation**	**Test ID**	**Test OOD**	**Test Long**
Number of examples	12,000	2,000	2,000	2,000	1,500

Each labeled JSON Lines (JSONL) contains the keys id, expression, value (the output value of the expression), length, operator_count, and depth.

In order to have a more comprehensive interpretation of the results, it can be useful to provide an overview of the training dataset.

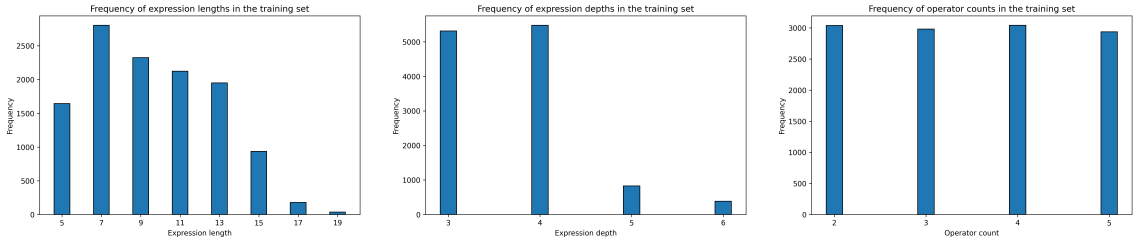


Figure 1: Distribution of the training dataset by expression length, depth, and operator count.

As we can observe, there is an absence of trivially short expressions such as `2+2`, however the number of expressions longer than 15 characters is very limited, and expressions longer than 19 characters are completely absent. Furthermore, while the expression depth is not uniformly distributed, the operator count appears to be more evenly balanced.

3 Models and preprocessing

We developed two different models: one based on a transformer architecture, and the other based on a convolutional neural network architecture.

3.1 Preprocessing

For both models, we created a vocabulary, containing the following characters: `0`, `1`, `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`, `+`, `-`, `(`, `)`, `<PAD>`, `<UNK>`, `[CLS]`. While the first characters are those contained in the expression, the additional characters, `<PAD>`, `<UNK>`, and `[CLS]`, have the following purposes: `<PAD>` is the padding character, used to make all expressions have the same length, `<UNK>` represents unknown characters, and `[CLS]` is used for classification (as explained in Section 3, only for the transformer model). The data processed by the transformer model is a tokenized string, which is made of a `[CLS]` token, followed by the tokenized expression and by padding tokens if necessary. The data processed by the CNN model is a tokenized string, containing the tokenized expression and the padding tokens if necessary. The tokens were obtained by assigning an integer value to each character in the vocabulary. Furthermore, for the transformer model, we created a subset of the training set for each feature (length, depth, operator count). These training sets contained expressions with the corresponding feature less than or equal to a fixed value where this value was selected among the possible values of the feature. The convolutional neural network model worked on the whole dataset instead.

3.2 The Transformer model structure

In the development of the first model, we used PyTorch to construct the TransformerCalculator class (which extends the torch.nn.Module class). This class implements the transformer structure, together with the positional encoding technique (using another support class), which allows the model to distinguish the positions of the tokens (e.g. we want the model to process differently "4-5" and "5-4"). The tokenized expression processed by the instance of the TransformerCalculator class contains a special token, the `[CLS]` token, which is used to predict the outcome. The idea is that the `[CLS]` token summarizes the meaning of the expression. A padding mask has been used to ignore the `<PAD>` tokens in the multi-head attention mechanism. At the end of the encoding process, the TransformerCalculator class instance takes the `[CLS]` token representation and passes it to the regression head, composed of a simple feed-forward neural network. This network maps the high-dimensional representation to a single scalar value, which is the predicted numerical result of the expression.

3.3 The Convolutional Neural Network model structure

The second model developed implements the architecture of convolutional neural networks (CNN). The CNN model formulates the task as a classification problem over the 199 possible output values in the range $[-99, 99]$. Instead of directly predicting a continuous scalar value, the model outputs a vector of logits, one for each possible result. This choice is particularly suitable for this task because the desired output is an integer belonging to a fixed and relatively small set of possible values.

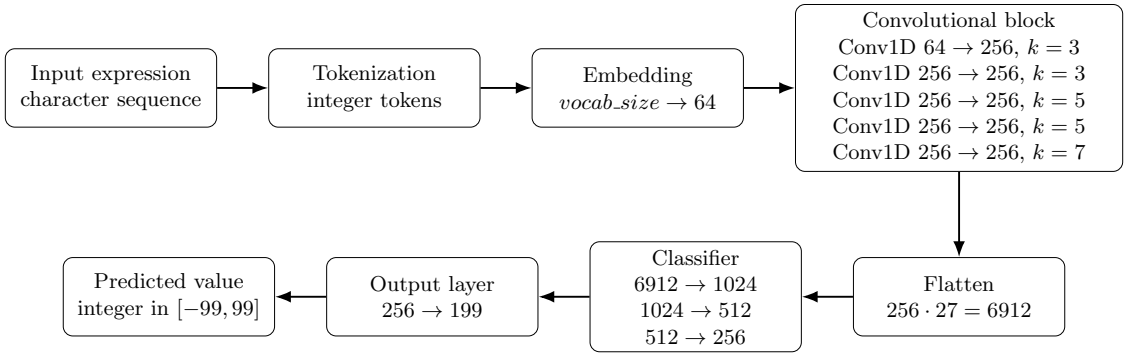


Figure 2: Compact architecture of the CNN calculator model.

Previously, other models had been taken into consideration; specifically, we tried to tackle the problem through fully connected neural networks (FCNN), obtaining however poor results, espe-

cially on the long test dataset, justified by the inability of FCNNs to recognize patterns in different positions of the input. The CNN architecture consists of five one-dimensional convolutional layers. The smaller kernels capture local symbolic patterns, while larger kernels allow the model to aggregate information over wider parts of the expression. ReLU activations provide non-linearity, and dropout is used as a regularization technique.

4 Training of the models

4.1 Training of the Transformer model

For the training loss, the transformer model used the MSE loss, since it penalizes large errors more strongly compared to other loss functions. The reason behind this choice was to highlight errors outside of the training distribution in order to have better predictions of results of expressions with unusual forms. For optimization of the parameters, we used the AdamW algorithm.

4.2 Training of the CNN model

The CNN was trained by minimizing the Cross-Entropy loss, with the model parameters optimized using AdamW. We use Cross-Entropy Loss because the CNN treats the task as a classification problem. The loss encourages the model to assign the highest probability to the class corresponding to the correct integer value.

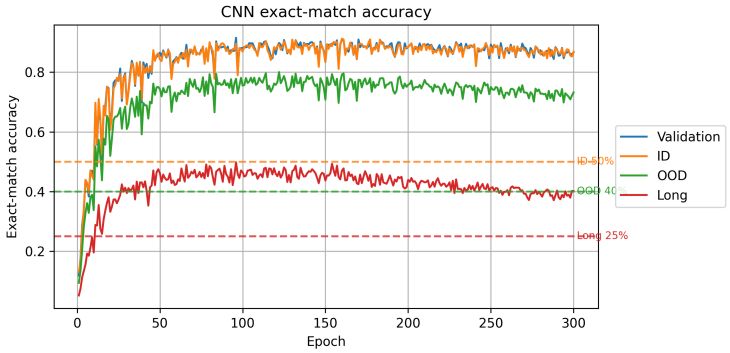


Figure 3: Training and validation accuracy as a function of the number of epochs.

4.3 Training challenges

For the transformer model, we adopted the following strategy to improve results on the long sequence test.

For each feature, the model was trained on a smaller subset of the training set (such subsets were first mentioned in Subsection 3.1). After a fixed number of steps, we moved to a larger subset, with higher values of the corresponding feature. For example, if length was chosen, the first thirty steps were trained on expressions in the training set with length less than or equal to 5. After that, we increased the threshold and repeated the process. The validation test was also done on a subset of the whole validation set, using the same strategy to create it.

The idea was that the model should first learn the basic structure of simpler expressions and then gradually move to more complex ones. In this way, during training, the model was exposed to expressions with increasing values of the selected feature, for example longer sequences in the case of length. This procedure was intended to improve the model's ability to generalize to expressions with shapes that were not seen during training.

Unfortunately, this strategy did not bring us to a significant improvement. The three trained model obtained did not reach the exact-match accuracy of the CNN model.

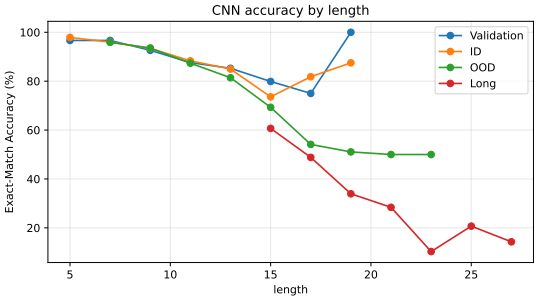
5 Analysis of result and conclusion

The transformer model performed worse than the convolutional neural network on the long-sequence test. This difference in performance may have been partly due to the different number of parameters: 540,417 for the Transformer model and 9,148,423 for the CNN model.

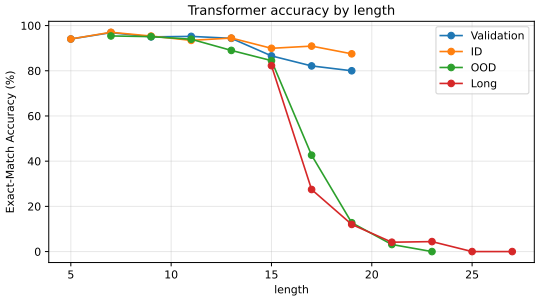
Split	CNN				Transformer			
	Acc. (%)	**MAE**	**MSE**	**RMSE**	**Acc. (%)**	**MAE**	**MSE**	**RMSE**
Validation	90.80	0.0985	0.1145	0.3383	94.35	0.1935	0.0682	0.2611
ID	90.60	0.1055	0.1535	0.3918	94.55	0.1954	0.0654	0.2557
OOD	79.30	0.3910	4.7330	2.1755	80.55	0.9619	8.2711	2.8759
Long	44.47	1.7480	27.0653	5.2024	36.80	4.1652	45.2546	6.7272

Table 1: Performance comparison between CNN and Transformer on the four evaluation splits.

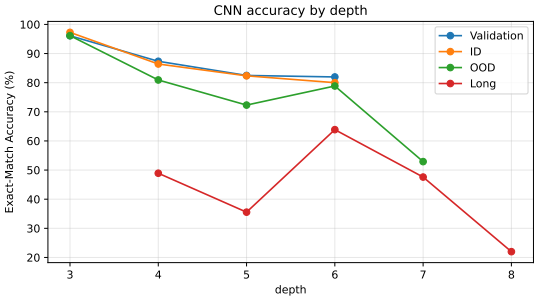
The next graphics illustrate how length, depth and operator count affect the exact-mach accuracy.



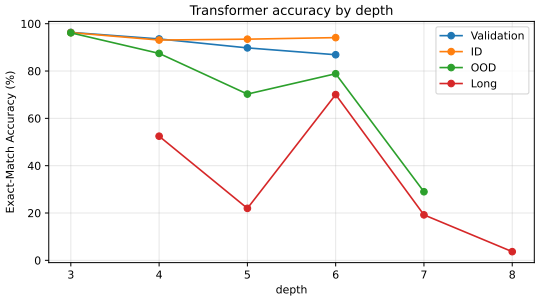
(a) CNN – Length



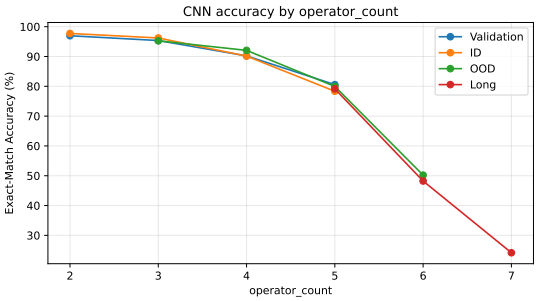
(b) Transformer – Length



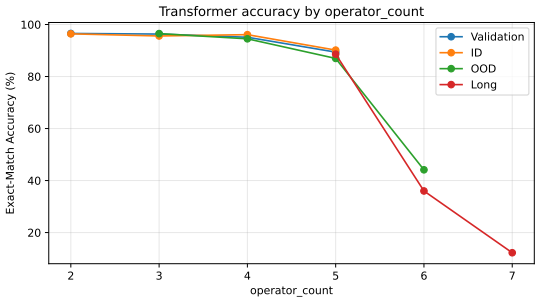
(c) CNN – Depth



(d) Transformer – Depth



(e) CNN – Operator count



(f) Transformer – Operator count

Figure 4: Exact-match accuracy of the CNN and Transformer models as a function of expression length, depth, and operator count.

As expected, expression length was an important factor affecting the accuracy of both models: once the length exceeds 15 characters, the accuracy generally starts to decrease, especially on the OOD and long-sequence splits. Similarly, the operator count also played a relevant role: both models maintained a high accuracy for expressions whose number of operators remained within the range observed in the training set. However, the same cannot be said for the depth feature, whose impact on accuracy seems to vary. This final observation could be partly explained by the different distribution of operator counts in the OOD and long-sequence splits.